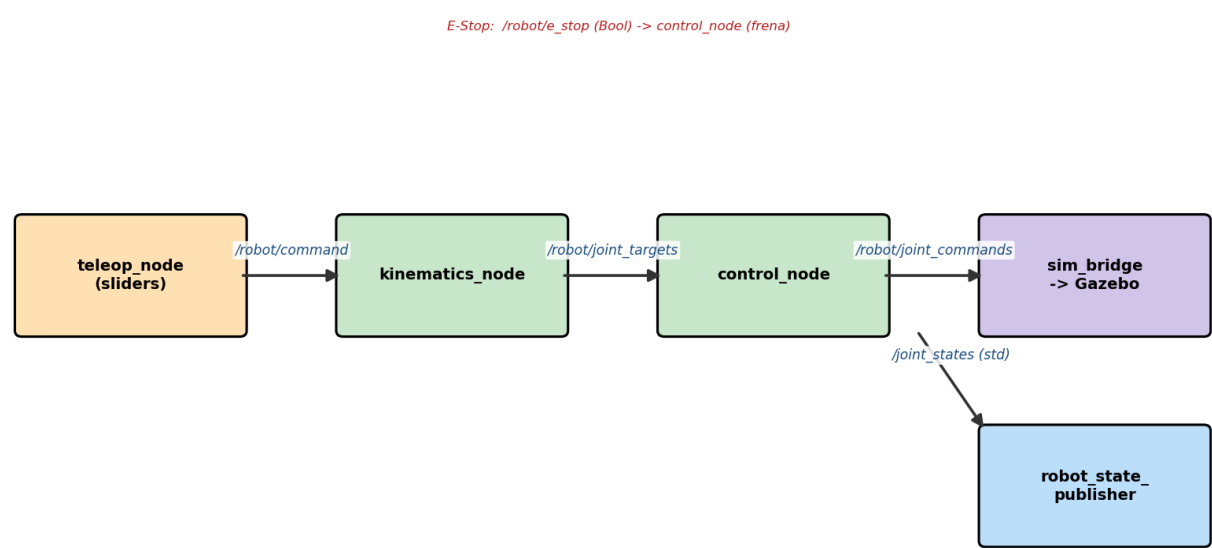


Robot Bípedo - Arquitectura (ROS 2 Humble)

Resumen visual de paquetes, nodos y tópicos. El operador mueve un slider; eso viaja por tópicos hasta mover la pata en RViz/Gazebo.

Flujo de comunicación del robot bípedo (ROS 2)



Nodos: quién publica, en qué tópico, y quién escucha

Publicador	Tópico	Suscriptor
teleop_node	/robot/command	kinematics_node
kinematics_node	/robot/joint_targets	control_node
control_node	/robot/joint_states	(estado interno del robot)
control_node	/robot/joint_commands	sim_bridge
control_node	/joint_states (std)	robot_state_publisher
nodo de seguridad (pendiente)	/robot/e_stop (Bool)	control_node (frena)
robot_state_publisher	/robot_description	(RViz / Gazebo)
robot_state_publisher	TF	(RViz / Gazebo)

Paquetes (interfaces del proyecto)

Paquete	Qué es / para qué sirve
robot_interfaces	Define los mensajes que usan todos los paquetes. Es el "contrato" de comunicación. No corre ningún nodo.
robot_description	URDF/Xacro: la forma física del robot (links, joints, límites). Sin algoritmos.
robot_kinematics	Nodo kinematics_node: recibe la orden y la convierte en consigna por motor.
robot_control	Nodo control_node: sigue la consigna con PID, aplica límite y E-Stop, publica el estado.
robot_simulation	Nodo sim_bridge: lleva el comando de motores a Gazebo (gemelo).
robot_teleop	Nodo teleop_node: sliders (una barra por motor) para mandar la orden. Requiere pantalla.
robot_bringup	Launch que prende todos los nodos juntos para demostrar la arquitectura.

Para probar

- 1) `cd ~/Documentos/Robot_Bipedo/ros2_ws && source /opt/ros/humble/setup.bash && colcon build && source install/setup.bash`
- 2) `ros2 launch robot_bringup bringup.launch.py`
- 3) `ros2 run rviz2 rviz2 -> Fixed Frame: base_link -> Add RobotModel -> Description Topic: /robot_description`